

סיכום מפורט של המודל, הנתונים והעיבוד — SmartECG

ECG Heartbeat Classification → SmartECG Assist **פרויקט:

Afeka — Machine Learning **מוסד:

(MIT-BIH Arrhythmia Database (PhysioNet **דאטה סט:

מודל: 1D CNN בינארי (Normal / Abnormal) **

תאריך: יוני 2026 **

תוכן עניינים

1. [סקירה כללית] (#1-סקירה כללית)
2. [המודל המקורי לעומת המודל הנוכחי] (#2-המודל המקורי לעומת המודל הנוכחי)
3. [דאטה סט MIT-BIH ומה עשינו לנתונים] (#3-דאטה סט mit-bih-ומה עשינו לנתונים)
4. [עיבוד מקדים (Preprocessing) — שלב אחר שלב] (#4-עיבוד מקדים-preprocessing--שלב אחר-שלב)
5. [חלוקה לאימון, ולידציה ומבחן] (#5-חלוקה לאימון-ולידציה-ומבחן)
6. [ארכיטקטורת המודל — 1D CNN] (#6-1D CNN-ארכיטקטורת המודל--1d-cnn)
7. [אימון, בחירת מודל ותוצאות] (#7-אימון-בחירת מודל-ותוצאות)
8. [מסקנות] (#8-מסקנות)

1. סקירה כללית

1.1 מטרת הפרויקט

לבנות מסווג אוטומטי לאות ECG (אק"ג) שמסווג כל **חלון זמן קצר** (שנייה אחת) כאחד משני המצבים:

תווית משמעות ערך מספרי
----- ----- -----
דופק תקין 0 **Normal**
דופק לא תקין (חריג) 1 **Abnormal**

1.2 זרימת העבודה המלאה

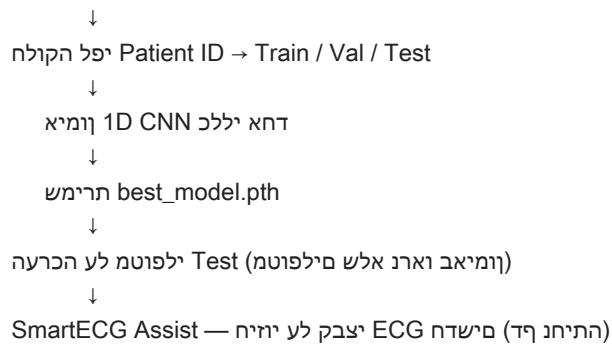
לפוטם לכל Annotations צבוק + CSV צבוק MIT-BIH:

↓

פילפוטם וניסו הניעט

↓

Preprocessing (resample, לומר, וניס) (Preprocessing)



1.3 עקרונות מרכזיים בגרסה הנוכחית

- **מודל אחד** — לכל המטופלים — לא מודל נפרד לכל אדם.
- **אימון מאפס (from scratch)** — ללא pre-training וללא fine-tuning per-patient.
- **חלוקה לפי מטופל** — מניעת דליפת מידע (data leakage).
- **סיווג בינארי** — פשוט, ברור, ומתאים לשימוש בדף הנחיתה.

2. המודל המקורי לעומת המודל הנוכחי

2.1 הגישה המקורית (לפני השינוי)

בגרסה הראשונית של הפרויקט, המערכת עבדה בגישת **מודל לכל מטופל (patient-specific)**:

רכיב הגישה המקורית

מספר מודלים מודל נפרד לכל מטופל ('individuals')
אימון ראשוני Pre-training על כל המטופלים יחד
התאמה אישית Fine-tuning נפרד לכל מטופל
ואימון שכבות לינאריות CNN ('get_frozen_cnn') הקפאת שכבות Transfer Learning
Grid Search Hyperparameters
חלוקת נתונים 'train_test_split' אקראי לפי beat בתוך כל מטופל
פלט תיקיית output נפרדת לכל מטופל

בעיה מרכזית בגישה המקורית:

הצדדים. זה יוצר **data leakage** — ביצועים מנופחים שלא משקפים יכולת התכללות למטופלים חדשים. כאשר beats מאותו מטופל מתפזרים בין Train ו-Test, המודל "רואה" את אותו אדם בשני

2.2 הגישה הנוכחית (לאחר השינוי)

| רכיב | הגישה הנוכחית |

| ----- | ----- |
| **מספר מודלים** | **מודל 1D CNN אחד** לכל המטופלים |
| **אימון** | **From scratch** — ללא pre-training וללא fine-tuning |
| בונה רשת חדשה בלבד | `generate_model()` — **הוסר** | **Transfer Learning** |
| הוצא משימוש Grid Search — (פרמטרים קבועים) (ברירת מחדל) | **Hyperparameters** |
| **חלוקת נתונים** | `split_patients_by_id()` — **לפי מזהה מטופל** |
| **מצב הרצה** | `general-training` (אימון) / `test` (הערכה) |
| **פלט** | תיקייה אחת: `/output/general_model` |

2.3 מה נשאר ללא שינוי

רכיב	סטטוס
ארכיטקטורת `cnn1D.py` (1D CNN)	ללא שינוי מבני
פונקציית `BCELoss + Sigmoid` Loss	
`resample 128`, נרמול, חלונות, `Bandpass`	**בסיסי `Preprocessing`**
**דאטה סט MIT-BIH	

2.4 מה השתנה בקוד

קובץ	שינוי
`general-training` נוסף; `fine_tuning` / `individuals` הוסרו לולאות	`main.py`
`get_frozen_cnn` ו-`checkpoint` הוסרו טעינת	`models_generation.py`
אימון והערכה כלליים; תיקייה אחת לפלט	`running.py`
`exclude_patients`, `load_patients_data`, `split_patients_by_id` נוספו	`data_processing.py`
(deprecated) הוצא משימוש	`grid_search.py`

2.5 למה ביצענו את השינוי?

1. **הערכה הוגנת** — בדיקה על מטופלים שלא נראו באימון משקפת שימוש אמיתי.
2. **פשטות** — מודל אחד קל יותר לתחזוקה, פריסה ושימוש בדף הנחיתה.
3. **מניעת overfitting למטופל** — במקום "לזכור" מטופל ספציפי, המודל לומד דפוסים כלליים.
4. **התאמה ל-SmartECG Assist** — משתמש חדש מעלה CSV בלי מודל מותאם אישית.

3. דאטה סט MIT-BIH ומה עשינו לנתונים

3.1 מקור הנתונים

- שם: MIT-BIH Arrhythmia Database
- מקור: <https://www.physionet.org/content/mitdb/1.0.0> — PhysioNet
- נתיב בפרויקט: `/dataset/mitbih\_database`

3.2 מבנה הקבצים

לכל מטופל (record ID) יש שני קבצים:

| קובץ | תוכן |

| ----- | ----- |

| `{id}.csv` | (ועמודות נוספות לעיתים) Lead MLI, רציף — עמודות: מספר דגימה ECG אות | `{id}.csv` |

| `{id}.annotations.txt` | beat — beat-by-beat תיוג | `{id}.annotations.txt` |

דוגמה לקובץ אות (`csv.100`):

```
'sample #','MLII','V5'  
0,995,1011  
1,995,1011  
...
```

3.3 פרמטרים טכניים של הדאטה

| פרמטר | ערך |

| ----- | ----- |

| קצב דגימה | 360 Hz |

| מספר מטופלים בדאטה המקורי | 48 |

| משך רשומה טיפוסית | 30~ דקות למטופל |

| גודל כולל של הדאטה | 471 MB |

3.4 מיפוי תיוגים לסיווג בינארי

Normal → 0:

- `N` — Normal beat

Abnormal → 1:

- `L` — Left bundle branch block beat
- `R` — Right bundle branch block beat
- `V` — Premature ventricular contraction
 - `A`, `a` — Atrial premature beat
- `j`, `S` — Supraventricular premature beat

- 'F' — Fusion of ventricular and normal beat
- 'E', 'e' — Ventricular escape beat
- 'r', '!', 'f', '/' — beats נוספים סוגי
- **נזרקים (לא משמשים לאימון):**
- 'Q', '?' — לא מסווגים / לא ברורים

3.5 סינון מטופלים (Patient Exclusion)

לפני האימון, מטופלים עם **חוסר איזון קיצוני** מוצאים מהדאטה:

קריטריוני הוצאה ('exclude_patients'):

1. מטופל עם **מחלקה אחת בלבד** (רק Normal או רק Abnormal).

2. מטופל שבו פחות מ- **0.9%** מהחלונות מסווגים כ-Abnormal.

תוצאה:

• **48** מטופלים בדאטה המקורי

• **19** מטופלים הוצאו

• **29** מטופלים תקפים לאימון

רשימת 19 המטופלים שהוצאו:

101, 103, 107, 109, 111, 112, 113, 115, 117, 118, 121, 122, 123, 124, 207, 214, 230, 232, 234

**למה זה חשוב?*

מטופל עם 99.9% beats תקינים גורם למודל "לנחש תמיד Normal" בלי ללמוד דפוסים אמיתיים של חריגות.

3.6 יחידת הדגימה לאימון — חלון (Window)

לא מאמנים על אות שלם של 30 דקות. כל מטופל מפוצל ל**חלונות של שנייה אחת**:

• שנייה אחת = **360 דגימות** (ב-360 Hz)

• כל חלון = **דגימת אימון אחת** עם תווית 0 או 1

• מטופל אחד → בערך **1,800–1,500 חלונות**

• 29 מטופלים → **עשרות אלפי דגמאות** לאימון

4. עיבוד מקדים (Preprocessing) — שלב אחר שלב

כל העיבוד מתבצע ב-`src/data\_processing.py` (אימון) וב-`src/inference.py` (דף נחיתה).

באימון יש גם שימוש ב-annotations; בדף הנחיתה — רק על האות, ללא תיוג.

4.1 שלב 1 — טעינת האות והתיוגים

```
signal = pd.read_csv("{patient}.csv") # תוא ECG
annotations = pd.read_fwf("{patient}annotations.txt") # סיגנית
```

- נלקחת עמודת **Lead II** (**MLII**) כאות הראשי.
- לכל **beat** יש מיקום (sample number) וסוג (A, V, N...).

4.2 שלב 2 — Bandpass Filter (סינון תדרים)

```
| פרמטר | ערך |
| ----- | ---- |
| סוג מסנן | Butterworth Bandpass |
| תדר נמוך (low cut) | 0.4 Hz |
| תדר גבוה (high cut) | 30 Hz |
| קצב דגימה | 360 Hz |
| סדר המסנן (order) | 3 |
| יישום | 'filtfilt' (סינון דו-כיווני, ללא הזזת פאזה) |

**מטרה:**
```

- הסרת **baseline wander** (תנועת קו בסיס איטית, מתחת ל-0.4 Hz)
- הסרת רעשים בתדרים גבוהים (מעל 30 Hz)
- שמירה על רכיבי ה-QRS הרלוונטיים לזיהוי דופק

4.3 שלב 3 — Normalization (נרמול)

```
• **Min-Max normalization** [0, 1]
• מחושב **בנפרד לכל מטופל** (לא גלובלי על כל הדאטה)

normalized = (signal - min(signal)) / (max(signal) - min(signal))
```

****למה per-patient**?**

יכולים להיות בגודל משרעת שונה. נרמול פר-מטופל מאפשר למודל להתמקד ב**צורת הגל** ולא בערך מוחלט. מטופלים שונים

4.4 שלב 4 — Segmentation (חלוקה לחלונות)

האות מחולק לחלונות של **360 דגימות** (= שנייה אחת):

```
פיצור תוא: |---|---|---|---|
תנועה: [0][1][2][3]...
תומיגד 360 360 360 360
```

****:**(`split\_to\_windows`) כל חלון ****:**

1. לכל דגימה בחלון — מסומן 0 (Abnormal), 1 (Normal), או 1- (לא מסווג).

2. תיוג החלון = ******מקסימום ****** התיוגים בתוכו (`np.argmax`).

- אם יש `beat abnormal` אחד לפחות → החלון = ******(Abnormal (1

- אם רק 0 (Normal → ******(Normal

- אם אין תיוג → החלון ******נזרק

******דוגמה:

1: N beats קר → Normal (0)
2: N + V beat → Abnormal (1)
3: annotation אלל → קרנ

4.5 שלב 5 — Resampling (360 → 128)

כל חלון של 360 דגימות עובר ******דגימה מחדש ****** ל-128 נקודות:

`resample(window_360_samples, 128)`

• משתמש ב- (`scipy.signal.resample` (FFT-based).

• ******אותה שנייה בזמן ****** — רק פחות נקודות.

• 128 = אורך הקלט שה-CNN מצפה לו (`input_size`).

(לדומל טלק) תדוקן 128 → (הובג טוריפ) תדוקן 360
תחא היינש תחא היינש

4.6 שלב 6 — Reshape לקלט CNN

$(N, 128) \rightarrow (N, 1, 128)$

• ******N ****** = מספר החלונות

• ******1 ****** = ערוץ אחד (single-channel 1D signal)

• ******128 ****** = אורך החלון

4.7 סיכום שרשרת Preprocessing

| שלב | קלט | פלט | מטרה |

| ----- | ----- | ----- |

| טעינה | קבצי Series + DataFrame | CSV + annotations | קריאת נתונים |

| אות גולמי | אות מסונן | הסרת רעש | Bandpass |

| אות מסונן | [0, 1] | אחידות בין מטופלים | Normalization |

| אות מנורמל | חלונות 360 + תוויות | יחידות אימון | Segmentation |

| Resample | 360 | גודל קבוע ל | 128 דגימות | CNN-דגימות |
| Reshape | (N, 128) | (N, 1, 128) | PyTorch פורמט |

5. חלוקה לאימון, ולידציה ומבחן

5.1 עקרון מרכזי — חלוקה לפי מטופל

**** החלוקה מתבצעת לפי Patient ID — לא לפי beat בודד. ****

דבלב Train → ולש תונולחה לכ → 100 לפוטמ
דבלב Test → ולש תונולחה לכ → 119 לפוטמ

Train, Val ו-Test מאותו מטופל ****לעולם לא**** מתפזרים בין beats >

****למה? ****

הנחיתה), המודל מקבל אות ****מטופל חדש**** שלא נראה באימון. חלוקה לפי מטופל מדמה את התרחיש הזה.
בשימוש אמיתי (ודף

5.2 השוואה לגישה הישנה

| | גישה ישנה | גישה נוכחית |

| | ----- | --- |

| | יחידת חלוקה | Beat / חלון בודד | ****מטופל שלם**** |

| | דליפת מידע | כן — אותו מטופל ב-Train וב-Test | ****לא**** |

| | מדידת ביצועים | מנופחת, לא אמינה | משקפת התכללות אמיתית |

5.3 יחסי החלוקה

הפונקציה `split_patients_by_id()` מבצעת חלוקה דו-שלבית:

1. ****15%**** מהמטופלים → `test_ratio=0.15` ('Test')

2. מ-85% הנוותרים → ****~17.6%**** ל-Val (שווה ערך ל-15% מכלל המטופלים)

3. השאר → Train

| | יחס | מספר מטופלים | Split |

| | ----- | ---- |

| | ****Train**** | ~65.5% | 19 |

| | ****Validation**** | ~17.2% | 5 |

| | ****Test**** | ~17.2% | 5 |

| | ****סה"כ**** | 100% | 29 |

****פרמטרים:****

- `random\_state=42` (שחזוריות)
- `val\_ratio=0.15`, `test\_ratio=0.15`

5.4 רשימת מטופלים בכל Split

****Train (19 מטופלים):****

100, 105, 106, 108, 114, 116, 201, 202, 205, 208, 209, 212, 215, 219, 221, 222, 223, 228, 233

****Validation (5 מטופלים):****

102, 104, 200, 213, 217

****Test (5 מטופלים):****

119, 203, 210, 220, 231

החלוקה נשמרת בקובץ: `output/general\_model/patient\_splits.json`

5.5 איך נבנים מערכי האימון

לאחר החלוקה, חלונות מכל המטופלים בכל split ****מאוחדים (concatenate):****

```
x_train, y_train = concatenate_patient_splits(patient_data, train_patients)
```

```
x_val, y_val = concatenate_patient_splits(patient_data, val_patients)
```

```
x_test, y_test = concatenate_patient_splits(patient_data, test_patients)
```

****דוגמה לגודל Test set:****

• 5 מטופלים $\times$ $1,700 \sim$ חלונות במוצע $\approx 8,574$ ****חלונות**** בסך הכל

• כל חלון = דגימה אחת עם תווית 0/1

5.6 איזון מחלקות באימון

באימון מופעל ****Weighted Random Sampling**** (`weighted\_sampling=True`):

• מחלקות עם פחות דוגמאות (Abnormal) מקבלות ****משקל גבוה יותר****

• שיטה: oversampling — דגימה עם החזרה (replacement)

• מטרה: למנוע מהמודל להתעלם מחריגים בגלל חוסר איזון

ב-Validation וב-Test ****ללא**** oversampling (הערכה על התפלגות טבעית).

6. ארכיטקטורת המודל — 1D CNN

מוגדר ב-`src/models/cnn1D.py` — רשת קונבולוציה חד-ממדית בסגנון ****ResNet**** (skip connections).
המודל

6.1 קלט ופלט

```
| פריט | ערך |
| ----- | ----- |
| **Input shape** | `(batch_size, 1, 128)` |
| **Output** | (Sigmoid) הסתברות בין 0 ל-1 |
| **Threshold** |  $\geq 0.5 \rightarrow \text{Abnormal}$ ,  $< 0.5 \rightarrow \text{Normal}$  |
| **Loss** | Binary Cross Entropy (BCELoss) |
```

6.2 מבנה הרשת

```
Input: (1, 128)
↓
Conv1d: 1 → 32 channels, kernel_size=1
↓
ReLU
↓
× 4 ResNet-style Blocks:
  Conv1d(32→32, k=5) → ReLU
  Conv1d(32→32, k=5)
  + Skip Connection
  → ReLU → MaxPool1d(k=5, stride=2)
↓
Flatten → 160 features
↓
Linear: 160 → 160
↓
Linear: 160 → 1
↓
Sigmoid → P(Abnormal)
```

6.3 פרמטרי ארכיטקטורה

```
| פרמטר | ערך |
| ----- | ----- |
| `num_blocks` | 4 |
| `block_channels` | 32 |
| `kernel_size` | 5 |
| `num_features` (לאחר Flatten) | 160 |
```

6.4 חד-ממדי ResNet בלוק — Block

כל Block כולל:

1. Conv1d + ReLU

2. Conv1d

3. ****Skip connection**** (מתאים ממדים 1×1 Conv1d, חיבור קצר) — אם מספר ערוצים שונה)

4. ReLU

5. MaxPool1d — הקטנת אורך האות

7. אימון, בחירת מודל ותוצאות

7.1 הגדרות אימון

```
| פרמטר | ערך |
| ----- | ---- |
| Optimizer | Adam |
| Learning rate | 0.001 |
| LR Scheduler | ReduceLROnPlateau (factor=0.5, patience=5) |
| Weight decay | 1e-4 |
| Batch size | 32 |
| Epochs | 30 |
| Weighted sampling | True |
| Seed | 42 |
```

7.2 בחירת המודל הטוב ביותר

- בכל epoch — חישוב ****validation loss**** על מטופלי Val.
- נשמר `'best_model.pth'` כאשר val loss ****יורד**** (הנמוך ביותר).
- ****המודל שנבחר: validation loss = 0.4714**** — ****Epoch 5****
- לאחר epochs 30 — טעינת best model והערכה על מטופלי Test בלבד ******.

7.3 תוצאות על Test Set (מטופלים חדשים)

הערכה על ****8,574 חלונות**** מ-5 מטופלים שלא נראו באימון:

```
| מטריקה | ערך |
| ----- | ---- |
| **Accuracy** | 84.07% |
| **Balanced Accuracy** | 86.98% |
| **Recall** | 93.26% |
| **Precision** | 63.90% |
```

| ****Test Loss**** | 0.618 |

8.1 מה השגנו

1. **מודל 1D CNN כללי** לסיווג בינארי של אות ECG — Normal / Abnormal.
2. **שינוי מהותי** מגישת מודל-לכל-מטופל לגישת מודל אחד עם חלוקה הוגנת.
3. **Pipeline** → resample → עיבוד מלא: **סינון → נרמול → חלונות Pipeline**.
4. **84% accuracy 93% recall ו-84% accuracy** של test באימון על מטופלי.
5. **SmartECG Assist** — דף נחיתה לבדיקת קבצי ECG חדשים.

8.2 מגבלות

- המודל אומן על MIT-BIH בלבד — ביצועים על דאטה סטים אחרים עשויים להיות שונים.
- Train-Test בין Overfitting.
- רבים false positives — בינוני Precision.
- **לא מיועד לאבחון רפואי** — כלי לימודי והדגמה בלבד.

8.3 כיווני שיפור אפשריים

- ECG על חלונות Data augmentation
- Regularization (Dropout, Early Stopping)
- איזון מחלקות משופר (Focal Loss, SMOTE)
- אימון על דאטה סטים נוספים
- שיפור Precision תוך שמירה על Recall גבוה

מסמך זה נוצר עבור פרויקט SmartECG — Afeka, Machine Learning, יוני 2026.